UTF-8 (Full)

Table 8-6. *UTF-8 codepoint ranges*

First codepoint	Last codepoint	Byte 0	Byte 1	Byte 3	Byte 4	Byte 5	Byte 6
U+0000	U+007F	00–7F					
U+0080	U+07FF	C0–DF	80–BF				
U+0800	U+FFFF	E0–EF	80–BF	80–BF			
U+10000	U+1FFFF	F0–F7	80–BF	80–BF	80–BF		
U+200000	U+3FFFFFF	F8–FB	80–BF	80–BF	80–BF	80–BF	
U+4000000	U+7FFFFFFF	FC–FD	80–BF	80–BF	80–BF	80–BF	80–BF

The main reason why UTF-8 differs from FSS-UTF something called "self-synchronization": this means that overlapping code units do not form valid code units themselves. This prevents errors in transmission: all that's needed to work out the code unit boundaries is the stream of bytes itself, and this allows a program reading a UTF-8 byte stream to start at an arbitrary point and detect character boundaries correctly.

PHP and UTF-8

As stated above, PHP does not have "native" support for Unicode (the definition of which is debatable anyway), but it *does* have full support for UTF-8, both as data and as the encoding of PHP programs themselves. If you remember from earlier chapters, legal symbols for variable and function names are defined as follows:

```
[a-zA-Z_\x7f-\xff][a-zA-Z0-9_\x7f-\xff]*
```

This regular expression (we'll be having lots of fun with those soon, I promise) describes the uppercase and lowercase ASCII Latin alphabet (codepoints *0x41–0x5A* a–z and *0x61–0x7A* A–Z), the Arabic digits (codepoints *0x30–0x39*), and underscores (codepoint *0x5F*). It also allows for the extended ASCII (ISO-8859-1 et al.) range of *0x80–0xFF* (plus the final ASCII codepoint *0x7F*, which happens to be the DELETE control character). Therefore, any multi-byte UTF-8 string will meet the criteria for a